

Data Structures & Algorithms (AI) – ST1507

Assignment 1 (CA1)

Name: Goh Kun Ming

Student ID: P2415691

Class: DAAA/FT/2A/02

Lecturer: Liu Zheng

1. Introduction

Haikus are short poems traditionally comprising three lines that capture fleeting moments, usually related to nature or human emotions. This project involves building a haiku generator application called **Haikumator**, designed to process existing haikus and generate new variants using techniques such as synonym replacement, antonym inversion, compacting expressions (Zen-izing), and batch permutations. The assignment serves as a platform to apply object-oriented programming (OOP), custom data structures, and algorithm design.

2. User Guide & Demonstration

2.1. Starting of Application

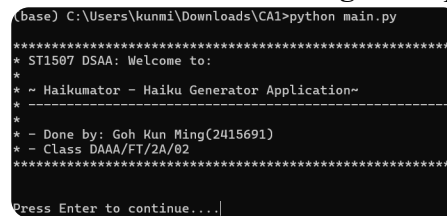
To run the **Haikumator Application**, the user must first open the **Anaconda Prompt**, navigate to the **Haikumator** folder, and enter the command shown in **Figure 1.1**.



```
python main.py
```

Figure 1.1: Initiation Command in Anaconda Prompt

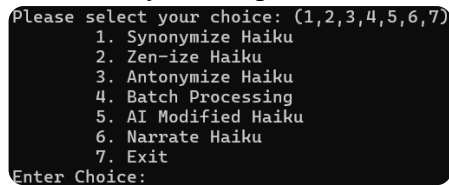
Upon executing the command with ‘Enter’, the **Welcome Page** is displayed (**Figure 1.2**).



```
(base) C:\Users\kunmi\Downloads\CA1>python main.py
*****
* ST1587 DSAA: Welcome to: *
* ~ Haikumator - Haiku Generator Application~ *
* ----- *
* - Done by: Goh Kun Ming(2415691) *
* - Class DAAA/FT/2A/02 *
*****
Press Enter to continue...|
```

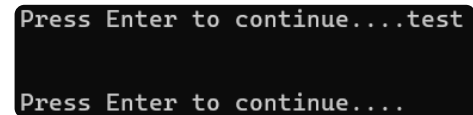
Figure 1.2: Welcome Page

Upon reaching the **Welcome Page**, the user is prompted to press ‘Enter’ to proceed to the **Selection Menu** (**Figure 1.3**). Any other input will result in a re-prompt, as shown in **Figure 1.4**. Only the ‘Enter’ key is accepted without triggering a re-prompt.



```
Please select your choice: (1,2,3,4,5,6,7)
1. Synonymize Haiku
2. Zen-ize Haiku
3. Antonymize Haiku
4. Batch Processing
5. AI Modified Haiku
6. Narrate Haiku
7. Exit
Enter Choice:
```

Figure 1.3: Selection Menu Figure

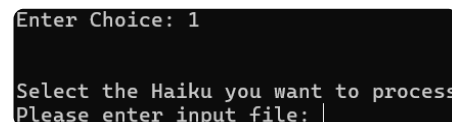


```
Press Enter to continue...test
Press Enter to continue....
```

1.4: Enter Key Re-Prompt

2.2 Option 1 – Synonymize Haiku

In the **Selection Menu**, entering ‘1’ prompts the user to input a **Haiku file** for the *Synonymize Haiku* operation (**Figure 2.1**).



```
Enter Choice: 1
Select the Haiku you want to process
Please enter input file: |
```

Figure 2.1: Haiku File Input

Upon entering a valid **Haiku file**, the user is prompted to input a **Synonym Thesaurus** (**Figure 2.2**). If a **non-existent file** is entered, a re-prompt appears, requesting a valid Haiku file (**Figure 2.3**). Similarly, entering a **Synonym or Antonym Thesaurus** at the Haiku prompt will also trigger a re-prompt (**Figure 2.4**).

```
Select the Haiku you want to process
Please enter input file: haiku_001.txt

Select a synonym thesaurus.
Please enter input file:
```

Figure 2.2: Synonym Thesaurus File Input

```
Select the Haiku you want to process
Please enter input file: test.txt
File not found. Please try again.

Select the Haiku you want to process
Please enter input file:
```

Figure 2.3: Non-existent File Re-Prompt

```
Select the Haiku you want to process
Please enter input file: haiku_001_synonyms.txt
That isn't a haiku file. Please select a haiku.

Select the Haiku you want to process
Please enter input file: |
```

Figure 2.4: Thesaurus File Re-Prompt

Upon entering a valid **Synonym Thesaurus**, the user is prompted to press ‘Enter’ to initiate the *Synonymize Haiku* operation (Figure 2.5); only ‘Enter’ is accepted (Figure 1.4). Invalid inputs, such as a **non-existent file** or a **Haiku text file**, trigger re-prompts (Figures 2.6 and 2.7, respectively).

```
Select a synonym thesaurus.
Please enter input file: haiku_001_synonyms.txt

Press Enter to continue....
```

Figure 2.5: Synonym Thesaurus File Input

```
Select a synonym thesaurus.
Please enter input file: test.txt
File not found. Please try again.

Select a synonym thesaurus.
Please enter input file:
```

Figure 2.6: Non-existent File Re-Prompt

```
Select a synonym thesaurus.
Please enter input file: haiku_001.txt
That isn't a thesaurus file. Please select a thesaurus.

Select a synonym thesaurus.
Please enter input file: |
```

Figure 2.7: Thesaurus File Re-Prompt

Pressing ‘Enter’ displays the **original** and **synonymized Haiku**, followed by another ‘Enter’ prompt to proceed to the **File Saving and Redo Option** (Figure 2.8).

```
the Haiku before processing:
-----
Last gust of autumn.
Red, yellow, brown unite.
Carpet under the trees

The Synonymized Haiku after processing:
-----
Ultimate blast of fall.
Ochre, tawny, auburn join.
Layer underneath the foliage

Press Enter to continue....
```

Figure 2.8: Before and After Processing

Pressing ‘Enter’ initiates the file-saving process (Figure 2.9); invalid inputs trigger a re-prompt (Figure 1.4). Inputting ‘y’ prompts for a file name (Figure 2.10). If the file exists, an overwrite confirmation appears (Figure 2.11); selecting ‘n’ returns to the name prompt (Figure 2.10). Only ‘y’ or ‘n’ are valid inputs.

```
Press Enter to continue....

Do you want to save the text file? y/n:
```

Figure 2.9: File Save Prompt

```
Do you want to save the text file? y/n: y
Please enter new filename:
```

Figure 2.10: File Name Prompt

```
Do you want to save the text file? y/n: y
Please enter new filename: haiku_001.txt

'haiku_001.txt' already exists. Overwrite? y/n:
```

Figure 2.11: Overwrite Confirmation

If the user declines to save (‘n’) or completes the save, pressing ‘Enter’ advances to the **Redo prompt** (Figure 2.12), where only ‘Enter’ is accepted. Choosing ‘y’ returns to the **Before and After Processing** screen with a new combination (Figure 2.8), while ‘n’ redirects to the **Selection Menu** (Figure 1.3). Only ‘y’ or ‘n’ are valid inputs.

```
Press Enter to continue....

Do you want to give this another try? y/n:
```

Figure 2.12: Redo Prompt

2.3 Option 2 – Zen-ize Haiku

In the **Selection Menu**, entering ‘2’ prompts the user to input a **Haiku file** for the *Zen-ize Haiku* operation (Figure 3.1).

```
Enter Choice: 2

Select the Haiku you want to process
Please enter input file:
```

Figure 3.1: Haiku Input Prompt

Upon entering a valid **Haiku file**, the user is prompted to input a **Synonym Thesaurus** (Figure 2.2). Invalid or non-existent files trigger re-prompts (Figures 2.3 and 2.4). Once a valid thesaurus is provided, the user must press ‘Enter’ to begin the synonymisation process (Figure 2.5); other inputs result in a re-prompt (Figure 1.4).

If the thesaurus input is invalid-either non-existent or a Haiku file-a re-prompt is issued (Figures 2.6 and 2.7). Pressing ‘Enter’ then displays the **original** and **synonymised Haiku**, followed by another ‘Enter’ prompt leading to the **File Saving and Redo Option** screen (Figure 3.2).

```

Press Enter to continue...

The Haiku before processing:
-----
Brilliance in the ocean.
Exclusively glitters for me.
My ship diverts, no more.

The Zen-ized Haiku after processing:
-----
Light in the deep.
Only gleams for me.
My ship turns, no more.

Press Enter to continue...

```

Figure 3.2: Before and After Processing

Pressing ‘Enter’ prompts the file-saving process (Figure 2.9); other inputs trigger a re-prompt (Figure 1.4). Entering ‘y’ requests a file name (Figure 2.10), with overwrite confirmation if the file exists (Figure 2.11). Declining returns the user to the name prompt. Only ‘y’ or ‘n’ are accepted. If the file is saved or declined (‘n’), pressing ‘Enter’ advances to the **Redo prompt** (Figure 2.12). Choosing ‘y’ restarts the process with a new combination (Figure 2.8); ‘n’ returns to the **Selection Menu** (Figure 1.3). Only valid inputs are accepted to proceed.

2.4 Option 3 – Antonymize Haiku

In the **Selection Menu**, entering ‘3’ prompts the user to input a **Haiku file** for the *Synonymize Haiku* operation (Figure 4.1).

```

Enter Choice: 3

Select the Haiku you want to process
Please enter input file:

```

Figure 4.1: Haiku File Input

Upon entering a valid **Haiku file**, the user is prompted to input a **Synonym Thesaurus** (Figure 2.2). Invalid or non-existent files trigger re-prompts (Figures 2.3 and 2.4). Once valid, the user is then prompted to input an **Antonym Thesaurus** (Figure 4.2), with the same validation conditions applied. Pressing ‘Enter’ begins the *Antonymisation* process (Figure 4.3); other inputs trigger a re-prompt (Figure 1.4).

```

Select the Haiku you want to process
Please enter input file: haiku_003.txt

Select a synonym thesaurus.
Please enter input file: haiku_003_synonyms.txt

Select an antonym thesaurus.
Please enter input file:

```

Figure 4.2: Antonym Thesaurus Input

```

Select the Haiku you want to process
Please enter input file: haiku_001.txt

Select a synonym thesaurus.
Please enter input file: haiku_001_synonyms.txt

Select an antonym thesaurus.
Please enter input file: haiku_001_antonyms.txt

Press Enter to continue...

```

Figure 4.3: Antonym Confirmation

If the thesaurus input is invalid-either non-existent or a Haiku file-a re-prompt is issued (**Figures 2.6 and 2.7**). Pressing ‘Enter’ displays the **original** and **antonymized Haiku**, followed by another ‘Enter’ prompt leading to the **File Saving and Redo Option** screen (**Figure 4.4**).

```

The Haiku before processing:
-----
Last gust of autumn.
Red, yellow, brown unite.
Carpet under the trees

The Antonymized Haiku after processing:
-----
First silence of spring.
Khaki, azure, light disperse.
Covering above the wildlife

Press Enter to continue....

```

Figure 4.4: Before and After Processing

Pressing ‘Enter’ prompts the file-saving process (**Figure 2.9**); other inputs trigger a re-prompt (**Figure 1.4**). Entering ‘y’ requests a file name (**Figure 2.10**), with overwrite confirmation if the file exists (**Figure 2.11**). Declining returns the user to the name prompt. Only ‘y’ or ‘n’ are accepted. If the file is saved or declined (‘n’), pressing ‘Enter’ advances to the **Redo prompt** (**Figure 2.12**). Choosing ‘y’ restarts the process with a new combination (**Figure 2.8**); ‘n’ returns to the **Selection Menu** (**Figure 1.3**). Only valid inputs are accepted to proceed.

2.5 Option 4 – Batch Processing

In the **Selection Menu**, entering ‘4’ prompts the user to input a **Haiku file** for the *Batch Processing Haiku* operation (**Figure 5.1**).

```

Enter Choice: 4

Select the Haiku you want to process
Please enter input file:

```

Figure 5.1: Haiku File Input

Upon entering a valid **Haiku file**, the user is prompted to input a **Synonym Thesaurus** (**Figure 2.2**). Invalid or non-existent files trigger re-prompts (**Figures 2.3 and 2.4**). Once valid, the user is then prompted to input an **Existing Folder** (**Figure 5.2**), only existing folder input is accepted. Pressing ‘Enter’ begins the *batch processing* (**Figure 5.3**); other inputs trigger a re-prompt (**Figure 1.4**).

```

Select the Haiku you want to process
Please enter input file: haiku_001.txt

Select a synonym thesaurus.
Please enter input file: haiku_003_synonyms.txt

Select an existing folder to store the generated haiku files
Please enter the folder name:

```

Figure 5.2: Existing Folder Prompt

```

Select an existing folder to store the generated haiku files
Please enter the folder name: HAIKU_003_SYNONYMIZED

Press Enter to continue....

```

Figure 5.3: Enter to Proceed

Pressing ‘Enter’ displays the **Batch Processing of Haiku**, followed by another ‘Enter’ prompt leading to the **File Saving and Redo Option** screen (**Figure 5.4**).

```

Press Enter to continue....

Batch processing started!
.....
Batch processing completed with 12 permutations

Press Enter to continue....

```

Figure 5.4: Batch Processing

Pressing ‘Enter’ will bring the user back to the **Selection Menu** (**Figure 1.3**).

2.6 Option 5 – AI Modified Haiku

In the **Selection Menu**, entering ‘5’ prompts the user to input a **Haiku file** for the *AI-Modified Haiku* operation (**Figure 6.1**).


```
Enter Choice: 5

AI Haiku Generator

Enter haiku training file:
```

Figure 6.1: Haiku File Input

Upon entering a valid **Haiku file**, the user is prompted to press ‘Enter’ to proceed with the AI-modified Haiku generation, which utilises a **Markov Probabilistic Model** and incorporates **fallback words** for added variety (Figure 6.2).

```
Enter haiku training file: haiku_001.txt

Press Enter to continue....
```

Figure 6.2: Enter Prompt

Upon pressing ‘Enter’, the user is shown the **AI-generated Haiku** (Figure 6.3), constructed using the **input Haiku training data** and **fallback words**.

The **AI-Modified Haiku** feature utilises a **Markov Chain-based language model** to generate text by predicting word sequences based on learned statistical patterns from a haiku corpus (Mitzenmacher & Upfal, 2005). To ensure adherence to haiku structure-**5-7-5 syllables**-a **custom syllable counter** enforces line-specific syllable limits.

When suitable word transitions are unavailable, a **fallback mechanism** selects from a predefined list of generic vocabulary, ensuring outputs remain complete, coherent, and stylistically appropriate. Additional **randomness** is introduced to avoid redundancy, including logic to prevent the first and third lines from being identical.

This **hybrid approach**, combining probabilistic modelling with rule-based constraints, aligns with best practices in AI design (Russell & Norvig, 2020). For users, this ensures **greater variety**, **consistent structure**, and a **more creative and reliable haiku-generation experience**, even from minimal input data.

```
AI-Modified Haiku
-----
Still dew river leaf.
Brown glow dream mist drift still dew.
Leaf space fade leaf fade

Press Enter to continue....|
```

Figure 6.3: AI-Modified Haiku

Pressing ‘Enter’ prompts the file-saving process (Figure 2.9); other inputs trigger a re-prompt (Figure 1.4). Entering ‘y’ requests a file name (Figure 2.10), with overwrite confirmation if the file exists (Figure 2.11). Declining returns the user to the name prompt. Only ‘y’ or ‘n’ are accepted. If the file is saved or declined (‘n’), pressing ‘Enter’ advances to the **Redo prompt** (Figure 2.12). Choosing ‘y’ restarts the process with a new combination (Figure 2.8); ‘n’ returns to the **Selection Menu** (Figure 1.3). Only valid inputs are accepted to proceed.

2.7 Option 6 – Narrate Haiku

In the **Selection Menu**, entering ‘6’ prompts the user to input a **Haiku file** for the *Narrate Haiku* operation (Figure 7.1).

```
Enter Choice: 6

Enter the haiku file to narrate:
```

Figure 7.1: Haiku File Input

Upon entering a valid **Haiku file**, the user is prompted to press '**Enter**' to proceed with the line-by-line narration of the input Haiku File (**Figure 7.2**).

```
Enter the haiku file to narrate: haiku_001.txt
Press Enter to continue...
```

Figure 7.2: Enter Prompt

Upon pressing '**Enter**', narration begins immediately, progressing **line-by-line** through the input Haiku (**Figure 7.3**). Upon completion, the user is prompted to choose whether to **save the audio file**.

```
Haiku to Narrate
-----
Last gust of autumn.
Red, yellow, brown unite.
Carpet under the trees

Line 1: Last gust of autumn.
Line 2: Red, yellow, brown unite.
Line 3: Carpet under the trees

Save audio to file? y/n: |
```

Figure 7.3: Haiku Narrating

Entering '**y**' requests a file name (**Figure 2.10**), with overwrite confirmation if the file exists (**Figure 2.11**). Declining returns the user to the name prompt. Only '**y**' or '**n**' are accepted.

If the file is saved or declined ('**n**'), pressing '**Enter**' advances to the **Redo prompt** (**Figure 2.12**). Choosing '**y**' restarts the process with a new combination (**Figure 2.8**); '**n**' returns to the **Selection Menu** (**Figure 1.3**). Only valid inputs are accepted to proceed.

2.8 Option 7 – Exit

In the **Selection Menu**, entering '**7**' terminates the program (**Figure 8**).

```
Enter Choice: 7

Bye, thanks for using ST1507 DSAA: Haikumator
```

Figure 8: Exit Program

3. Object-Oriented Programming (OOP) Approach

3.1 Class Overview

Table 1: Classes Used and Responsibility

<u>Class Name</u>	<u>Responsibility</u>
TokenizedLine	Processes and tokenizes individual haiku lines.
Thesaurus	Manage synonyms and antonym lookups and formatting.
Menu	Handle main menu output and interface.
KeywordIndex	Builds and maintains keyword-index mappings.
InputValidator	Ensures user input is valid and handles re-prompts.
Haiku	Core controller managing all haiku transformations.
FileHandler	Handles file input/output operations.
HaikuAudioNarrator	Use audio data to narrate and save haikus as audio.
SyllableCounter	Determine the number of syllables of word to ensure 5-7-5 format.
MarkovChainModel	A bigram-based Markov chain model to generate haiku lines with syllable constraints.
AIHaikuGenerator	Implements syllable-aware Markov chain-based haiku generation.

3.2 OOP Concepts

- **Encapsulation:** Each class encapsulates its functionality (e.g. `AIHaikuGenerator` encapsulates Markov logic; `FileHandler` handles I/O).
- **Abstraction:** Users interact with simplified methods like `generate_haiku()` or `process()` without dealing with underlying complexity.
- **Polymorphism:** Classes such as `Haiku`, `AIHaikuGenerator`, and `Thesaurus` implement similar method names (`process`, `train_model`) with different behaviours.
- **Inheritance:** While not deeply used, the structure supports extendable base classes (e.g. `BaseProcessor` for synonym/zen/antonym processors).
- **Composition:** The `Haiku` class uses objects like `InputValidator`, `FileHandler`, and `AIHaikuGenerator`, demonstrating “has-a” relationships.
- **Reusability:** Classes like `TokenizedLine`, `SyllableCounter`, and `Thesaurus` are reused across features (traditional and AI).
- **Modularity:** Code is organised across separate files, supporting maintainability and scalability.

3.3 Class Diagram

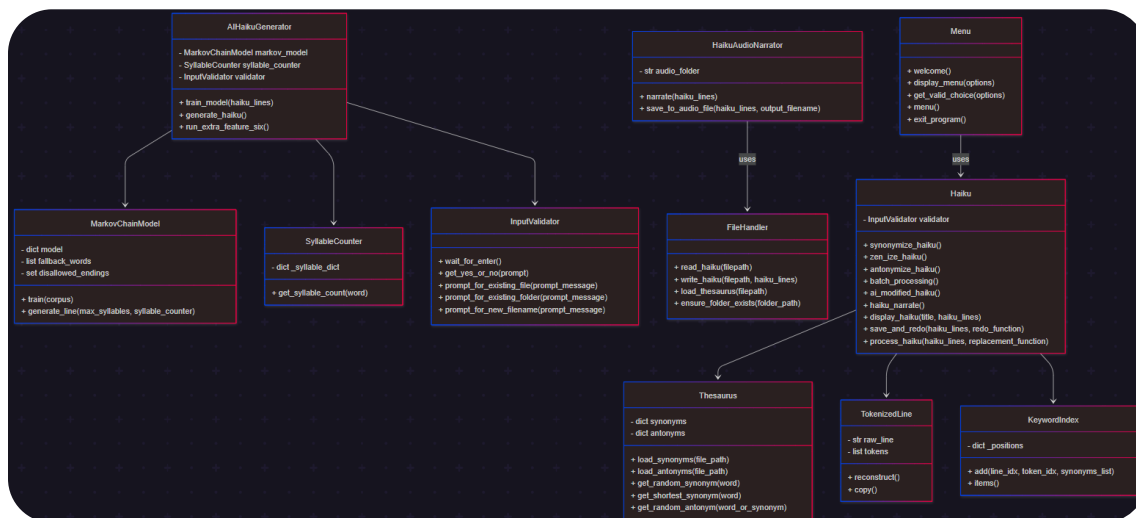


Figure 9: Class Diagram

4. Data Structures & Algorithms

4.1 Algorithms Used

- **Synonym Replacement:** Linear scan + dictionary lookup.
Complexity: $O(n)$, where n is number of tokens.
- **Zen-izing:** Linear scan with min-length selection from synonym list.
Complexity: $O(n * k)$, where k is number of synonyms.
- **Antonym Replacement:** Requires mapping synonym back to keyword before looking up antonym.
Complexity: $O(n * k)$
- **Batch Processing:** Cartesian product generation using `itertools.product`.
Complexity: $O(s_1 * s_2 * \dots * s_n)$ where s_i is number of synonyms for each keyword.

4.2 Data Structures Used

Table 2: Data Structures Used

Structure	Type	Usage in Code	Built-in / Custom	Time Complexity
dict	Hash Table	Thesaurus mappings, syllable counts, Markov transitions	Built-in	O(1)
list	Dynamic Array	Token sequences, haiku lines, synonyms/antonyms lists	Built-in	O(1) (indexing)
set	Hash Set	Disallowed word endings, uniqueness checks	Built-in	O(1)
TokenizedLine	Tuple Array	Represents words as (prefix, core, suffix) for reconstruction	Custom	O(n)
KeywordIndex	Map of Tuples	Maps token positions to synonym sets for batch generation	Custom	O(n)

5. Challenges & Learning Points

5.1 Challenges Faced

- Designing modular OOP architecture without use of collections library.
- Ensuring fallbacks and randomness still produced meaningful haikus.
- Preventing poor haiku endings (e.g., ending in articles like "the").
- Managing file operations and user input validation without breaking flow.
- Conceptualising extra features that are both unique and technically feasible (e.g., AI-generated haikus and audio narration).
- Refining and justifying ideas using theoretical sources, such as using the classic "AI: A Modern Approach" textbook to guide Markov Chain design and hybrid modelling principles.

5.2 Learning Points

- Mastery of OOP principles under constraints.
- Use of fallback mechanisms for AI text generation.
- Practical application of Big-O in managing batch combinatorics.
- Enhanced creativity and planning in ideating usable, value-adding extra features under technical and grading constraints.
- Exposure to how traditional AI theory can inform and support modern generative systems, even in creative applications.
- Designing for extensibility and robustness.

6. Code Appendix

6.1 Source Code of 'ai_generator.py'

```

from ai_syllables import SyllableCounter
from ai_markov import MarkovChainModel
from file_handler import FileHandler
from input_utils import InputValidator

class AIHaikuGenerator:
    """
    Generates haikus using an AI-enhanced Markov chain and syllable-aware constraints.

    This class handles:
    - Training on haiku datasets
    - Generating haikus in 5-7-5 format
    - Ensuring line uniqueness and correct syllable structure
    - Saving and narrating generated output with user interaction

    Methods:
    - train_model(): Train the Markov chain using provided haikus
    - generate_haiku(): Generate a single 3-line haiku
    - run_extra_feature_six(): Full user-facing loop with file loading, saving,
and regeneration
    """

    # ===== #
    # ===== AIHaikuGenerator Class Initialisation ===== #
    # ===== #

    def __init__(self):
        """
        Initialise the AI haiku generator with required components.

        Components:
        - `MarkovChainModel`: For haiku generation based on bigram transitions.
        - `SyllableCounter`: For tracking and validating syllable counts per word.
        - `InputValidator`: For handling file I/O and user interaction.

        Attributes:
        markov_model (MarkovChainModel): The trained word transition model.
        syllable_counter (SyllableCounter): Utility for syllable-aware line
generation.
        validator (InputValidator): Handles user prompts and input validation.
        """
        self.markov_model = MarkovChainModel()
        self.syllable_counter = SyllableCounter()

```

```

        self.validator = InputValidator()

# ===== #
# ===== Train Markov Chain AI ===== #
# ===== #

def train_model(self, haiku_lines: list[str]) -> None:
    """
    Train the internal Markov chain model using the provided haiku lines.

    This method delegates training to the `MarkovChainModel` instance,
    enabling it to learn word transitions from the dataset.

    Args:
        haiku_lines (list[str]): A list of haiku lines used as training data.

    Returns:
        None
    """
    # Train the Markov model using the given haiku corpus
    self.markov_model.train(haiku_lines)

# ===== #
# ===== Generate New Haiku ===== #
# ===== #

def generate_haiku(self) -> list[str]:
    """
    Generate a 3-line haiku with a 5-7-5 syllable structure.

    This method:
    - Generates each line using the Markov model and syllable counter.
    - Ensures the third line is not a repeat of the first (up to 3 retries).
    - Adds proper punctuation to the first two lines for readability.

    Returns:
        list[str]: A list containing the three haiku lines.
    """
    # Generate first and second lines with 5 and 7 syllables
    first = self.markov_model.generate_line(5, self.syllable_counter)
    second = self.markov_model.generate_line(7, self.syllable_counter)

    # Ensure the third line is not a duplicate of the first
    attempts = 0
    while True:
        third = self.markov_model.generate_line(5, self.syllable_counter)
        if third != first or attempts >= 3:

```

```

        break
    attempts += 1

    # Format: full stop on line 1 and 2, clean ending on line 3
    return [
        first.rstrip(".") + ".",
        second.rstrip(".") + ".",
        third.rstrip(".")
    ]

# ===== #
# ===== Run AI Generator (Markov) ===== #
# ===== #

def run_extra_feature_six(self):
    """
    Run the AI-powered haiku generation workflow for Extra Feature Six.

    This method:
    - Prompts the user to load a training haiku file.
    - Trains the Markov model using the input.
    - Generates and displays a 3-line AI haiku.
    - Allows the user to save the result and optionally regenerate another.

    Returns:
        None
    """
    print("\n\n=== AI Haiku Generator ===")

    # Prompt for training input file
    path = self.validator.prompt_for_existing_file("Please enter the training
haiku file: ")
    haiku_lines = FileHandler.read_haiku(path)

    # Train the model using the file contents
    self.train_model(haiku_lines)

    try:
        # Generate the haiku and display it
        result = self.generate_haiku()
        print("\nAI-Generated Haiku")
        print("-" * 30)
        for line in result:
            print(line)

        # Ask to save the generated haiku
        self.validator.wait_for_enter()

```

```

        if self.validator.get_yes_or_no("Would you like to save the haiku? y/n: ")
== "y":
            filename = self.validator.prompt_for_new_filename("Enter filename to
save: ")

            FileHandler.write_haiku(filename, result)
            print(f"Haiku saved to '{filename}'.")

            # Offer to regenerate another haiku
            self.validator.wait_for_enter()
            if self.validator.get_yes_or_no("Generate another haiku? y/n: ") == "y":
                return self.run_extra_feature_six()

    except Exception as e:
        print(f"Failed to generate haiku: {e}")

```

6.2 Source Code of 'ai_markov.py'

```

import random

class MarkovChainModel:
    """
    A bigram-based Markov chain model to generate haiku lines with syllable
constraints.

    This class:
    - Trains on a list of haiku lines (corpus) to build word transition probabilities.
    - Uses fallback words when the chain cannot continue.
    - Ensures haiku lines do not end with weak grammar words (e.g., "the", "in").
    - Supports syllable-count-limited generation for poetic rhythm.

    Attributes:
        model (dict[str, list[str]]): Markov chain structure mapping words to next
possible words.
        fallback_words (list[str]): Generic poetic words used when the model runs out
of transitions.
        disallowed_endings (set[str]): Words not permitted at the end of haiku lines.
    """

    # ===== #
    # ===== MarkovChainModel Class Initiation ===== #
    # ===== #

    def __init__(self):
        """
        Initialise the Markov chain model with an empty transition map
and predefined fallback vocabulary for haiku generation.

```

```

    Attributes:
        model (dict[str, list[str]]): Stores word transitions for bigram
generation.
        fallback_words (list[str]): Generic poetic words used when no transition
is found.
        disallowed_endings (set[str]): Grammar words that should not end a haiku
line.
    """
    # Markov transition dictionary: word → list of possible next words
    self.model = {}

    # Generic fallback vocabulary to inject creativity and avoid chain dead-ends
    self.fallback_words = [
        "the", "sun", "moon", "river", "mist", "breeze", "echo",
        "sky", "cloud", "light", "stone", "dream", "leaf", "wind",
        "dew", "dawn", "peace", "still", "stream", "glow", "twilight",
        "longing", "drift", "dance", "fade", "forest", "shade", "petal",
        "whisper", "soft", "bloom", "air", "space", "dust", "shimmer"
    ]

    # Weak grammar words not allowed as final word in a haiku line
    self.disallowed_endings = {
        "the", "a", "an", "of", "and", "in", "on", "at", "by", "to", "for",
"with", "from", "as"
    }

    # ===== #
    # ===== Train Markov Chain Model ===== #
    # ===== #

    def train(self, corpus: list[str]) -> None:
        """
        Train the Markov chain model using a corpus of haiku lines.

        This method builds a bigram transition dictionary where each word maps
        to a list of possible successor words, based on the given training data.

        Args:
            corpus (list[str]): A list of haiku lines used to train the model.

        Returns:
            None
        """
        for line in corpus:
            words = line.strip().split()

```



```

        # Iterate through word pairs to populate the model
        for i in range(len(words) - 1):
            current = self._clean_word(words[i])
            next_word = words[i + 1]

            if current not in self.model:
                self.model[current] = []

            self.model[current].append(next_word)

# ===== #
# ===== Generate Haiku Line ===== #
# ===== #

def generate_line(self, max_syllables: int, syllable_counter) -> str:
    """
    Generate a haiku line with a syllable limit using the trained Markov chain.

    This method attempts up to 15 times to generate a valid haiku line
    that respects the specified syllable count. If no valid line is generated,
    it returns a fallback message.

    Args:
        max_syllables (int): Maximum number of syllables allowed in the line.
        syllable_counter: An object that provides a `get_syllable_count(word)`
method.

    Returns:
        str: A generated haiku line, or a failure message if generation is
unsuccessful.
    """
    for _ in range(15): # Try multiple attempts to meet the syllable target
        line = self._attempt_line(max_syllables, syllable_counter)
        if line:
            return line

    return "[Failed to generate haiku line]"

# ===== #
# ===== Attempt Haiku Generating ===== #
# ===== #

def _attempt_line(self, max_syllables, syllable_counter):
    """
    Attempt to generate a haiku line within the syllable limit.

    This method starts with a random word and iteratively adds the next word

```

using the Markov model and fallback vocabulary. It ensures the line ends on a meaningful word by removing weak grammatical endings.

Args:

max_syllables (int): Maximum allowed syllables for the line.

syllable_counter: An object with a `get\_syllable\_count(word)` method.

Returns:

str: A haiku line that fits the syllable constraint, or an empty string if failed.

"""

Start with a random word (from model or fallback)

word = random.choice(list(self.model.keys()) + self.fallback_words)

words = [word]

sylls = syllable_counter.get_syllable_count(word)

Build the line until reaching the syllable limit

while sylls < max_syllables:

Get possible next words and shuffle for randomness

next_candidates = self.model.get(self._clean_word(word), []) +
self.fallback_words

random.shuffle(next_candidates)

Try each candidate until one fits within syllable limit

for candidate in next_candidates:

clean_candidate = self._clean_word(candidate)

next_sylls = syllable_counter.get_syllable_count(clean_candidate)

if sylls + next_sylls <= max_syllables:

words.append(candidate)

sylls += next_sylls

word = candidate

break

else:

No suitable word found; exit loop

break

Ensure the final word is not a weak grammar word

while self._clean_word(words[-1]) in self.disallowed_endings and len(words) >

1:

words.pop()

return self._final_format(words)

=====

===== Remove Punctuations of Word =====

=====

```

def _clean_word(self, word):
    """
    Clean a word by converting it to lowercase and stripping common punctuation.

    This helps ensure consistent lookup in the Markov chain and syllable
    dictionary.

    Args:
        word (str): The word to clean.

    Returns:
        str: The cleaned word with punctuation removed.
    """
    return word.lower().strip(".,!?:\\'")

# ===== #
# ===== Final Word Restriction ===== #
# ===== #

def _final_format(self, words: list[str]) -> str:
    """
    Format the final haiku line by capitalising the first word
    and joining all words into a single string.

    This method ensures the line begins with a capital letter and
    spacing is consistent. No punctuation is added beyond spacing.

    Args:
        words (list[str]): List of words forming the haiku line.

    Returns:
        str: A properly formatted haiku line, or an empty string if no words were
    given.
    """
    if not words:
        return ""

    # Capitalise the first word
    words[0] = words[0].capitalize()

    # Join words into a sentence-like line
    return " ".join(words)

```

6.3 Source Code of 'ai_syllables.py'

```
class SyllableCounter:
    """
    A utility class to manage syllable counts for words used in haiku generation.

    This class provides:
    - A built-in syllable dictionary for common haiku words.
    - Interactive fallback prompting for unknown words.
    - Caching of user-provided syllable counts to avoid repeated prompts.

    Attributes:
        _syllable_dict (dict[str, int]): Maps lowercase words (without punctuation)
                                         to their syllable counts.
    """

    # ===== #
    # ===== SyllableCounter Class Initaition ===== #
    # ===== #

    def __init__(self):
        """
        Initialise the syllable counter with a predefined dictionary.

        The dictionary includes syllable counts for a wide range of words
        commonly found in nature-themed and abstract haikus.
        """

        self._syllable_dict = {
            "last": 1,
            "first": 1,
            "tender": 2,
            "newest": 2,
            "fresh": 1,
            "young": 1,
            "gust": 1,
            "stillness": 2,
            "silence": 2,
            "serenity": 4,
            "calmness": 2,
            "tranquility": 4,
            "autumn": 2,
            "spring": 1,
            "springtime": 2,
            "march": 1,
            "april": 2,
            "may": 1,
            "red": 1,
```

```
"green": 1,  
"greenish": 2,  
"khaki": 2,  
"emerald": 3,  
"jade": 1,  
"yellow": 2,  
"blue": 1,  
"azure": 2,  
"cobalt": 2,  
"skyblue": 2,  
"sapphire": 2,  
"brown": 1,  
"teal": 1,  
"navy": 2,  
"silver": 2,  
"light": 1,  
"grey": 1,  
"unite": 2,  
"separate": 3,  
"split": 1,  
"part": 1,  
"scatter": 2,  
"disperse": 2,  
"carpet": 2,  
"roof": 1,  
"ceiling": 2,  
"crown": 1,  
"canopy": 3,  
"covering": 3,  
"under": 2,  
"above": 2,  
"overhead": 3,  
"upstairs": 2,  
"atop": 2,  
"beyond": 2,  
"trees": 1,  
"flowers": 2,  
"grass": 1,  
"creatures": 2,  
"fauna": 2,  
"wildlife": 2,  
"final": 2,  
"of": 1,  
"the": 1,  
"ultimate": 3,  
"latest": 2,  
"finishing": 3,
```

```
"closing": 2,  
"breeze": 1,  
"puff": 1,  
"breath": 1,  
"blast": 1,  
"flurry": 2,  
"fall": 1,  
"equinox": 3,  
"october": 3,  
"november": 3,  
"december": 3,  
"crimson": 2,  
"rust": 1,  
"ochre": 2,  
"reddish": 2,  
"magenta": 3,  
"tawny": 2,  
"gold": 1,  
"yellowish": 3,  
"pale": 1,  
"ashen": 2,  
"auburn": 2,  
"tan": 1,  
"brownish": 2,  
"bronze": 1,  
"swarthy": 2,  
"join": 1,  
"fuse": 1,  
"merge": 1,  
"blend": 1,  
"amalgamate": 4,  
"rug": 1,  
"mat": 1,  
"layer": 2,  
"blanket": 2,  
"spread": 1,  
"below": 2,  
"beneath": 2,  
"underneath": 3,  
"down": 1,  
"downunder": 3,  
"bushes": 2,  
"foliage": 3,  
"shrubs": 1,  
"plants": 1,  
"vegetation": 4,  
"brilliance": 2,
```

```
"radiance": 3,  
"sunlight": 2,  
"splendour": 2,  
"ocean": 2,  
"deep": 1,  
"water": 2,  
"exclusively": 4,  
"utterly": 3,  
"solely": 2,  
"only": 2,  
"specially": 3,  
"just": 1,  
"uniquely": 3,  
"glitters": 2,  
"sparkles": 2,  
"gleams": 1,  
"shimmers": 2,  
"glistens": 2,  
"diverts": 2,  
"redirects": 3,  
"turns": 1,  
"in": 1,  
"for": 1,  
"me": 1,  
"my": 1,  
"ship": 1,  
"no": 1,  
"more": 1,  
"cats": 1,  
"felines": 2,  
"kittens": 2,  
"basking": 2,  
"lying": 2,  
"relaxing": 3,  
"idling": 2,  
"lit": 1,  
"hit": 1,  
"reach": 1,  
"late": 1,  
"afternoon": 1,  
"rays": 1,  
"sun": 1,  
"not": 1,  
"all": 1,  
"wall": 1,  
"sky": 1,  
"storm": 1,
```



```
"cloud": 1,  
"clouds": 1,  
"leaves": 1,  
"blow": 1,  
"float": 1,  
"drift": 1,  
"moss": 1,  
"river": 2,  
"stream": 1,  
"whispers": 2,  
"echo": 2,  
"path": 1,  
"trail": 1,  
"branch": 1,  
"bark": 1,  
"dawn": 1,  
"dusk": 1,  
"twilight": 2,  
"horizon": 3,  
"glow": 1,  
"shade": 1,  
"peace": 1,  
"quiet": 2,  
"song": 1,  
"hum": 1,  
"tree": 1,  
"dew": 1,  
"still": 1,  
"longing": 2,  
"dance": 1,  
"fade": 1,  
"forest": 2,  
"petal": 2,  
"whisper": 2,  
"soft": 1,  
"air": 1,  
"space": 1,  
"dust": 1,  
"shimmer": 2,  
"moon": 1,  
"on": 1,  
"at": 1,  
"a": 1,  
"rain": 1,  
"dark": 1,  
"dream": 1,  
"flow": 1,
```

```

        "stone": 1,
        "silent": 2,
        "falling": 2,
        "glimmer": 2,
        "morning": 2,
        "evening": 3,
        "shadows": 2,
        "tranquil": 2,
        "fluttering": 3,
        "eternal": 3,
        "drifting": 2,
        "bloom": 1,
        "mist": 1,
        "leaf": 1,
        "peaceful": 2,
        "flicker": 2,
        "wind": 1,
    }

# ===== #
# ===== Obtain Syllable Count of Word ===== #
# ===== #

def get_syllable_count(self, word: str) -> int:
    """
    Retrieve or prompt for the syllable count of a given word.

    If the syllable count is already cached, it is returned directly.
    Otherwise, the user is prompted to input a positive integer count,
    which is then cached for future lookups.

    Args:
        word (str): The word to get the syllable count for.

    Returns:
        int: The number of syllables in the word.
    """
    # Normalise the word by stripping punctuation and converting to lowercase
    clean_word = word.lower().strip(".,;:?!\\'")

    # Return cached syllable count if available
    if clean_word in self._syllable_dict:
        return self._syllable_dict[clean_word]

    # Prompt the user until a valid positive integer is provided
    while True:
        try:

```

```

        count = int(input(f"\nHow many syllables does '{clean_word}' have?
    ").strip())

    if count <= 0:
        raise ValueError

    # Cache the result and return it
    self._syllable_dict[clean_word] = count
    return count

except ValueError:
    print("Invalid input. Please enter a positive integer.")

```

6.4 Source Code of ‘audio_narrator.py’

```

import os
import time
import wave
import winsound

class HaikuAudioNarrator:
    """
    A class that handles audio narration of haikus using pre-recorded `.wav` files.

    This class supports:
    - Narrating haiku line-by-line by playing each word's audio file.
    - Concatenating individual word audio into a single `.wav` file.
    - Gracefully skipping missing audio while notifying the user.

    Attributes:
        audio_folder (str): Path to the directory containing word-level `.wav` files.
    """

    # ===== #
    # ===== HaikuAudioNarrator Class Initiation ===== #
    # ===== #

    def __init__(self, audio_folder: str = "audio_data"):
        """
        Initialise the HaikuAudioNarrator with a specified folder for audio clips.

        Args:
            audio_folder (str): Path to the folder containing `.wav` files for
narration.

            Defaults to "audio_data".
        """
        # Directory containing individual word audio files

```

```

        self.audio_folder = audio_folder

# ===== #
# ===== Obtain Audio Data ===== #
# ===== #

def _get_wav_path(self, word: str) -> str:
    """
    Construct the file path for the `.wav` file corresponding to the given word.

    This method:
    - Converts the word to lowercase.
    - Strips leading/trailing punctuation commonly found in haikus.
    - Constructs the full path to the `.wav` file inside the audio folder.

    Args:
        word (str): The word to convert into an audio file path.

    Returns:
        str: Full path to the expected `.wav` file for the word.
    """
    clean = word.lower().strip(".,!?")
    return os.path.join(self.audio_folder, f"{clean}.wav")

# ===== #
# ===== Play Text by Text ===== #
# ===== #

def _play_word(self, word: str) -> None:
    """
    Play the `.wav` audio file corresponding to the given word.

    This method:
    - Constructs the path to the word's audio file.
    - Plays the sound if the file exists.
    - Otherwise, notifies the user that the word is missing.
    - Adds a small delay after playing for smooth audio flow.

    Args:
        word (str): The word to play as audio.

    Returns:
        None
    """
    filepath = self._get_wav_path(word)

    if os.path.exists(filepath):

```

```

        winsound.PlaySound(filepath, winsound.SND_FILENAME)
    else:
        print(f"[Missing: {word}] ", end='')

    # Short delay between words
    time.sleep(0.05)

# ===== #
# ===== Narrate Haiku File ===== #
# ===== #

def narrate(self, haiku_lines: list[str]) -> None:
    """
    Narrate a haiku line-by-line by playing each word's corresponding `.wav` file.

    This method:
    - Iterates over each line in the haiku.
    - Prints the line number and text to the console.
    - Plays each word's audio clip using `_play_word()`.
    - Pauses briefly between lines for pacing.

    Args:
        haiku_lines (list[str]): A list of haiku lines to narrate audibly.

    Returns:
        None
    """
    for i, line in enumerate(haiku_lines):
        print(f"\nLine {i + 1}: {line}")

        # Play each word individually
        for word in line.strip().split():
            self._play_word(word)

        # Small pause between lines
        time.sleep(0.2)

# ===== #
# ===== Save Audio File ===== #
# ===== #

def save_to_audio_file(self, haiku_lines: list[str], output_filename: str) ->
None:
    """
    Save the haiku as a single stitched `.wav` file by concatenating individual
    word audio clips.

```

This method:

- Iterates through each word in the haiku.
- Loads corresponding `.wav` files from disk.
- Concatenates all valid audio streams into a single `.wav` file.
- Skips any word with missing audio and notifies the user.

Args:

haiku_lines (list[str]): The haiku lines to narrate and save.

output_filename (str): Desired output filename (with or without .wav extension).

Returns:

None

"""

Ensure correct file extension

output_path = output_filename if output_filename.endswith(".wav") else
output_filename + ".wav"

try:

with wave.open(output_path, 'wb') as output:

params_set = False

Iterate over each word in the haiku

for line in haiku_lines:

for word in line.strip().split():

word_path = self._get_wav_path(word)

if not os.path.exists(word_path):

print(f"[Missing: {word}] - skipping")

continue

with wave.open(word_path, 'rb') as wf:

Set audio parameters only once

if not params_set:

output.setparams(wf.getparams())

params_set = True

Append audio frames to output

output.writeframes(wf.readframes(wf.getnframes()))

print(f"Haiku Audio saved to '{output_path}'")

except Exception as e:

print(f"Failed to save audio: {e}")

6.5 Source Code of 'file_handler.py'

```
import os

class FileHandler:
    """
    A utility class for handling file and folder operations in the Haikumator
    application.

    This class provides static methods to:
    - Read haiku lines from text files.
    - Write haiku lines to output text files.
    - Load thesaurus files (synonyms and antonyms) into dictionaries.
    - Ensure folders exist for saving outputs.

    All methods are static and can be used without instantiating the class.

    Typical usage includes:
    - File I/O for haiku transformations
    - Loading keyword mappings for synonym/antonym replacement
    - Ensuring output directories are valid before saving
    """

    # ===== #
    # ===== Read Input Haiku ===== #
    # ===== #

    @staticmethod
    def read_haiku(filepath):
        """
        Read haiku lines from a text file and return them as a list of strings.

        Each line in the file is stripped of leading/trailing whitespace.
        This method assumes the file contains exactly three lines of haiku.

        Args:
            filepath (str): Path to the haiku file.

        Returns:
            list[str]: A list of haiku lines, each as a separate string.
        """
        with open(filepath, 'r') as f:
            return [line.strip() for line in f.readlines()]

    # ===== #
    # ===== Write Haiku to Text File ===== #
    # ===== #
```

```

@staticmethod
def write_haiku(filepath, haiku_lines):
    """
    Write the given haiku lines to a text file, one line per row.

    Each line in the list is written to the file followed by a newline character.
    This method overwrites the file if it already exists.

    Args:
        filepath (str): The destination path where the haiku will be saved.
        haiku_lines (list[str]): A list of haiku lines (typically 3) to write to
the file.

    Returns:
        None
    """
    with open(filepath, 'w') as f:
        for line in haiku_lines:
            f.write(line + '\n')

# ===== #
# ===== Load Input Thesaurus ===== #
# ===== #

@staticmethod
def load_thesaurus(filepath):
    """
    Load a thesaurus file into a dictionary mapping keywords to word lists.

    Each line in the file must be formatted as:
        keyword: word1, word2, word3, ...

    This method strips whitespace and ensures all entries are stored cleanly
    for use in synonym or antonym lookups.

    Args:
        filepath (str): Path to the thesaurus text file.

    Returns:
        dict: A dictionary where each key is a keyword (str) and each value is a
list of associated words (list[str]).
    """
    thesaurus = {}

    with open(filepath, 'r') as f:
        for line in f:
            # Only process valid keyword: value lines

```



```

        if ':' in line:
            keyword, values = line.strip().split(":")
            thesaurus[keyword.strip()] = [v.strip() for v in
values.split(",")]

    return thesaurus

# ===== #
# ===== Validate Folder Exist ===== #
# ===== #

@staticmethod
def ensure_folder_exists(folder_path):
    """
    Ensure that the specified folder exists; create it if it does not.

    This method is useful for ensuring that an output directory is available
    before attempting to write files into it. If the folder does not already
exist,
    it will be created along with any necessary parent directories.

    Args:
        folder_path (str): The full path of the folder to check or create.

    Returns:
        None
    """
    if not os.path.exists(folder_path):
        os.makedirs(folder_path)

```

6.6 Source Code of 'haiku_utils.py'

```

import os
from itertools import product

from thesaurus import Thesaurus
from file_handler import FileHandler
from input_utils import InputValidator
from tokenized_utils import TokenizedLine
from keyword_utils import KeywordIndex
from ai_generator import AIHaikuGenerator
from audio_narrator import HaikuAudioNarrator

class Haiku:
    """
    A class to represent the Haiku processing engine.

```

Provides methods for transforming haikus using synonyms, antonyms, shortest-synonym (zen), AI-based generation, and batch processing.

Attributes:

 validator (InputValidator): Utility class to handle user input validation.

"""

=====

===== Haiku Class Initiation =====

=====

def __init__(self):

 """

 Initialise the Haiku engine.

 This constructor sets up the core validator used throughout the application for user input validation, such as file existence checks, yes/no prompts, etc.

 Attributes:

 validator (InputValidator): An instance used to validate user input across operations.

 """

 # Instantiate input validator for user prompts and validation

 self.validator = InputValidator()

=====

===== Haiku File Validation =====

=====

def _is_haiku_file(self, path: str) -> bool:

 """

 Check if the file at the given path is a haiku file.

 A file is considered a haiku file if none of its lines contain a colon (':'), distinguishing it from thesaurus files which use colons to define word mappings.

 Args:

 path (str): Path to the file to validate.

 Returns:

 bool: True if the file appears to be a haiku file, False otherwise.

 """

 try:

 with open(path, 'r', encoding='utf-8') as f:

 # Return True only if no line contains a colon

```

        return not any(': ' in line for line in f)
    except OSError:
        # If the file can't be opened, treat it as invalid
        return False

# ===== #
# ===== Thesaurus File Validation ===== #
# ===== #

def _is_thesaurus_file(self, path: str) -> bool:
    """
    Check if the file at the given path is a thesaurus file.

    A file is considered a thesaurus file if at least one line contains a colon
    (':'),
    which indicates a mapping from a keyword to a list of related words (e.g.,
    synonyms or antonyms).

    Args:
        path (str): Path to the file to check.

    Returns:
        bool: True if the file appears to be a thesaurus file, False otherwise.
    """
    try:
        with open(path, 'r', encoding='utf-8') as f:
            # Return True if any line contains a colon
            return any(': ' in line for line in f)
    except OSError:
        # If the file cannot be opened, treat it as invalid
        return False

# ===== #
# ===== Prompt User for Haiku File ===== #
# ===== #

def _prompt_haiku_file(self, prompt: str) -> str:
    """
    Prompt the user repeatedly until a valid haiku file is provided.

    A valid haiku file is defined as a text file where none of the lines contain a
    colon (':'),
    which would otherwise indicate it's a thesaurus file.

    Args:
        prompt (str): The message to display when requesting the file path.

```

```

Returns:
    str: The validated path to a haiku file.
"""
while True:
    # Ask the user for a file path
    path = self.validator.prompt_for_existing_file(prompt)

    # Check if the file conforms to the haiku format (no ':' in lines)
    if self._is_haiku_file(path):
        return path

    # If invalid, inform the user and retry
    print("That isn't a haiku file. Please select a haiku.")

# ===== #
# ===== Prompt User for Thesaurus File ===== #
# ===== #

def _prompt_thesaurus_file(self, prompt: str) -> str:
    """
    Prompt the user repeatedly until a valid thesaurus file is provided.

    A valid thesaurus file is defined as a text file where each line contains a
    keyword followed by a colon and a list of related words (e.g., synonyms or
    antonyms).

    Args:
        prompt (str): The message to display when asking for the file path.

    Returns:
        str: The validated path to a thesaurus file.
    """
    while True:
        # Prompt for a file path from the user
        path = self.validator.prompt_for_existing_file(prompt)

        # Check if the file has valid thesaurus format (lines contain ':')
        if self._is_thesaurus_file(path):
            return path

        # If invalid, notify user and prompt again
        print("That isn't a thesaurus file. Please select a thesaurus.")

# ===== #
# ===== Prompt User to Save / Redo ===== #
# ===== #

```

```

def save_and_redo(self, haiku_lines: list[str], redo_function: callable) -> None:
    """
    Handle post-processing by offering options to save the haiku and redo the
    operation.

    This method:
    - Prompts the user to optionally save the current haiku to a file.
    - Catches and reports any file-saving errors.
    - Prompts the user to optionally repeat the transformation process.

    Args:
        haiku_lines (list[str]): The haiku lines that were generated or
        transformed.
        redo_function (callable): The function to re-execute if the user wants to
        redo the process.

    Returns:
        None
    """
    # Pause before continuing
    self.validator.wait_for_enter()

    # Ask if the user wants to save the haiku to a file
    save_choice = self.validator.get_yes_or_no("Do you want to save the text file?
y/n: ")
    if save_choice == "y":
        new_filename = self.validator.prompt_for_new_filename("Please enter new
filename: ")
        try:
            FileHandler.write_haiku(new_filename, haiku_lines)
            print(f'The text has been saved in "{new_filename}"')
        except Exception as err:
            print(f"Failed to save. Error: {err}")

    # Pause before continuing
    self.validator.wait_for_enter()

    # Ask if the user wants to redo the transformation
    redo_choice = self.validator.get_yes_or_no("Do you want to give this another
try? y/n: ")
    if redo_choice == "y":
        return redo_function()

    # ===== #
    # ===== Display of Haiku to User ===== #
    # ===== #

```

```

def display_haiku(self, title: str, haiku_lines: list[str]) -> None:
    """
    Display a haiku with a formatted banner title.

    This method prints a stylised title followed by the haiku lines,
    with each line printed on its own line for clarity.

    Args:
        title (str): The banner text to display above the haiku.
        haiku_lines (list[str]): A list of strings representing the lines of the
haiku.

    Returns:
        None
    """
    # Print the banner title
    print(f"\n\n{title}")
    print("-" * 30)

    # Print each haiku line
    for line in haiku_lines:
        print(line)

# ===== #
# ===== Load Haiku and Synonym File ===== #
# ===== #

def _load_haiku_and_synonyms(self) -> tuple[list[str], Thesaurus]:
    """
    Load a haiku file and a synonym thesaurus file provided by the user.

    This method prompts the user to input two valid files:
    - a haiku file containing three poetic lines,
    - a synonym thesaurus file containing mappings from keywords to lists of
synonyms.

    It then loads the haiku lines and populates a Thesaurus object with the
synonym mappings.

    Returns:
        tuple[list[str], Thesaurus]:
            A tuple containing a list of haiku lines and a Thesaurus object with
synonyms loaded.
    """
    # Prompt the user to select a valid haiku file
    haiku_file = self._prompt_haiku_file(
        "Select the Haiku you want to process\nPlease enter input file: "

```

```

    )

    # Prompt the user to select a valid synonym thesaurus file
    synonyms_file = self._prompt_thesaurus_file(
        "Select a synonym thesaurus.\nPlease enter input file: "
    )

    # Wait for user confirmation before continuing
    self.validator.wait_for_enter()

    # Read haiku lines from the selected file
    haiku_lines = FileHandler.read_haiku(haiku_file)

    # Load synonyms into a Thesaurus object
    thesaurus = Thesaurus()
    thesaurus.synonyms = FileHandler.load_thesaurus(synonyms_file)

    return haiku_lines, thesaurus

# ===== #
# ===== Load Haiku and Thesaurus ===== #
# ===== #

def _load_haiku_synonyms_antonyms(self) -> tuple[list[str], Thesaurus]:
    """
    Load a haiku file, a synonym thesaurus file, and an antonym thesaurus file.

    This method prompts the user to provide three valid files:
    - a haiku file containing three lines of poetry,
    - a synonym thesaurus file (keyword: list of synonyms),
    - an antonym thesaurus file (keyword: list of antonyms).

    It then loads the haiku and both thesauruses into memory.

    Returns:
        tuple[list[str], Thesaurus]:
            A list of haiku lines and a Thesaurus object with both synonyms and
            antonyms populated.
    """
    # Prompt the user to select a valid haiku file
    haiku_file = self._prompt_haiku_file(
        "Select the Haiku you want to process\nPlease enter input file: "
    )

    # Prompt the user to select a valid synonym thesaurus file
    synonyms_file = self._prompt_thesaurus_file(
        "Select a synonym thesaurus.\nPlease enter input file: "
    )

```

```

    )

    # Prompt the user to select a valid antonym thesaurus file
    antonyms_file = self._prompt_thesaurus_file(
        "Select an antonym thesaurus.\nPlease enter input file: "
    )

    self.validator.wait_for_enter()

    # Load haiku lines from file
    haiku_lines = FileHandler.read_haiku(haiku_file)

    # Load thesaurus entries into a Thesaurus object
    thesaurus = Thesaurus()
    thesaurus.synonyms = FileHandler.load_thesaurus(synonyms_file)
    thesaurus.antonyms = FileHandler.load_thesaurus(antonyms_file)

    return haiku_lines, thesaurus

# ===== #
# ===== Process Haiku Replacement ===== #
# ===== #

def process_haiku(self, haiku_lines: list[str], replacement_function: callable) -> list[str]:
    """
    Apply a replacement function on haiku keywords to produce a transformed haiku.

    This function processes each line of the haiku by tokenizing it, applying the
    given replacement function to each keyword (typically for synonym/antonym
    substitution),
    and reconstructing the transformed line.

    Args:
        haiku_lines (list[str]): The original haiku lines to process.
        replacement_function (callable): A function that takes a lowercase string
        and returns a replacement string (e.g., a synonym or antonym).

    Returns:
        list[str]: A list of haiku lines with tokens replaced according to the
        function.
    """
    processed = []

    for raw_line in haiku_lines:

```



```

    # Tokenize the haiku line into structured parts
    token_line = TokenizedLine(raw_line)

    # Iterate through each token: (prefix, core word, suffix)
    for idx, (prefix, core, suffix) in enumerate(token_line.tokens):
        # Apply the replacement function to the core word (lowercased)
        repl = replacement_function(core.lower())

        if repl:
            # Preserve original capitalisation, if any
            if core and core[0].isupper():
                repl = repl.capitalize()

            # Replace the token in the token list
            token_line.tokens[idx] = (prefix, repl, suffix)

    # Reconstruct and append the processed line
    processed.append(token_line.reconstruct())

return processed

# ===== #
# ===== Option 1: Synonymize Haiku ===== #
# ===== #

def synonymize_haiku(self):
    """
    Option 1: Create a new haiku by replacing keywords with random synonyms.

    This feature transforms an existing haiku into a new variation by substituting
    specific keywords with randomly selected synonyms from a thesaurus file.
    """
    # Prompt user to load a haiku and synonym thesaurus
    haiku_lines, thesaurus = self._load_haiku_and_synonyms()

    def perform_synonymization():
        """
        Replace keywords in the haiku with randomly selected synonyms.

        This method displays the original haiku, performs synonym substitution
        using the thesaurus, shows the transformed version, and optionally allows
        the user to save or redo the transformation.

        Returns:
            Optional[None]: Recursively re-executes if user opts to try again.
        """
        # Display the original haiku to the user

```

```

        self.display_haiku("The Haiku before processing:", haiku_lines)

        # Process haiku lines with random synonyms
        processed = self.process_haiku(haiku_lines, thesaurus.get_random_synonym)

        # Display the transformed (synonymized) haiku
        self.display_haiku("The Synonymized Haiku after processing:", processed)

        # Offer user option to save or redo the operation
        return self.save_and_redo(processed, perform_synonymization)

    return perform_synonymization()

# ===== #
# ===== Option 2: Zen-ize Haiku ===== #
# ===== #

def zen_ize_haiku(self):
    """
    Option 2: Create a compact haiku by replacing keywords with the shortest
    synonyms.

    This feature aims to produce a more minimalistic or succinct form of the
    original haiku
    by substituting each keyword with its shortest available synonym.
    """
    # Load the haiku and synonym thesaurus from the user
    haiku_lines, thesaurus = self._load_haiku_and_synonyms()

    def perform_zenization():
        """
        Perform minimalistic transformation by replacing keywords with the
        shortest synonyms.

        The method uses the thesaurus to find and replace keywords with their most
        concise
        synonyms, often resulting in a simpler and "zen-like" version of the
        haiku.

        Returns:
            Optional[None]: Recursively re-executes if user opts to retry.
        """
        # Show the original haiku
        self.display_haiku("The Haiku before processing:", haiku_lines)

        # Apply shortest-synonym replacement

```

```

        processed = self.process_haiku(haiku_lines,
thesaurus.get_shortest_synonym)

        # Show the transformed haiku
        self.display_haiku("The Zen-ized Haiku after processing:", processed)

        # Prompt to save and optionally redo
        return self.save_and_redo(processed, perform_zenization)

    return perform_zenization()

# ===== #
# ===== Option 3: Antonymize Haiku ===== #
# ===== #

def antonymize_haiku(self):
    """
    Option 3: Create a contrasting haiku by replacing keywords or their synonyms
    with random antonyms.

    This function prompts the user for a haiku, synonym thesaurus, and antonym
thesaurus.
    It then replaces key words in the haiku with randomly chosen antonyms,
allowing for
    more imaginative or ironic haiku reinterpretations.
    """
    # Load user-selected haiku file, synonym thesaurus, and antonym thesaurus
    haiku_lines, thesaurus = self._load_haiku_synonyms_antonyms()

    def perform_antonymization():
        """
        Perform antonym replacement on the loaded haiku.

        The method replaces relevant words in the haiku with antonyms derived
either
        from the keywords or from their synonyms using the loaded thesaurus.

        Returns:
            Optional[None]: Recursively re-executes if user opts to regenerate.
        """
        # Display the original haiku
        self.display_haiku("The Haiku before processing:", haiku_lines)

        # Replace keywords with antonyms using the thesaurus
        processed = self.process_haiku(haiku_lines, thesaurus.get_random_antonym)

        # Display the antonymized haiku

```

```

        self.display_haiku("The Antonymized Haiku after processing:", processed)

        # Prompt to save and optionally redo
        return self.save_and_redo(processed, perform_antonymization)

    return perform_antonymization()

# ===== #
# ===== Option 4: Batch Processing ===== #
# ===== #

def batch_processing(self):
    """
    Option 4: Generate all permutations of a haiku using synonyms
    and save each version as a file.

    This method prompts the user for a haiku file, a synonym thesaurus,
    and an output folder. It systematically replaces each keyword with its
    synonyms to create all possible haiku variations.
    """
    # Prompt for required input files and folder
    haiku_file = self._prompt_haiku_file(
        "Select the Haiku you want to process\nPlease enter input file: "
    )
    synonyms_file = self._prompt_thesaurus_file(
        "Select a synonym thesaurus.\nPlease enter input file: "
    )
    output_folder = self.validator.prompt_for_existing_folder(
        "Select an existing folder to store the generated haiku files\n"
        "Please enter the folder name: "
    )
    self.validator.wait_for_enter()

    # Load haiku and tokenize each line
    raw_lines = FileHandler.read_haiku(haiku_file)
    tokenized_lines = [TokenizedLine(line) for line in raw_lines]

    # Load thesaurus and initialize keyword index
    thesaurus = Thesaurus()
    thesaurus.synonyms = FileHandler.load_thesaurus(synonyms_file)

    index = KeywordIndex()
    for i, tline in enumerate(tokenized_lines):
        for j, (_, core, _) in enumerate(tline.tokens):
            if core.lower() in thesaurus.synonyms:
                index.add(i, j, thesaurus.synonyms[core.lower()])

```

```

# Abort if no keywords found for replacement
if not any(True for _ in index.items()):
    print("\nNo keywords found in the haiku that match the thesaurus.")
    return

print("\nBatch processing started!")

# Prepare combinations of synonyms
counter = 0
positions = list(index.items()) # [(line_idx, token_idx), synonyms]
synonym_lists = [syms for (_, syms) in positions]

# Generate all permutations
for combo in product(*synonym_lists):
    counter += 1

    # Deep copy original tokens to modify independently
    copied = [t.copy() for t in tokenized_lines]

    for k, ((line_idx, token_idx), _) in enumerate(positions):
        prefix, _, suffix = copied[line_idx].tokens[token_idx]
        replacement = combo[k]

        # Preserve original capitalisation
        original_word = copied[line_idx].tokens[token_idx][1]
        if original_word and original_word[0].isupper():
            replacement = replacement.capitalize()

        # Replace the token
        copied[line_idx].tokens[token_idx] = (prefix, replacement, suffix)

    # Reconstruct lines and save to output
    out_lines = [t.reconstruct() for t in copied]
    fname = f"v{counter}.txt"
    FileHandler.write_haiku(os.path.join(output_folder, fname), out_lines)
    print(".", end="", flush=True)

print(f"\nBatch processing completed with {counter} permutations")
self.validator.wait_for_enter()

# ===== #
# ===== Option 5: AI-Modified Haiku ===== #
# ===== #

def ai_modified_haiku(self):
    """
    Option 5: Generate a new haiku using an AI-based trigram Markov model

```

with syllable constraints.

This feature uses an AI-enhanced generator trained on user-provided haiku data.

The generator produces a three-line poem that respects syllable rules and randomness.

```
"""
```

```
print("\n\nAI Haiku Generator")
```

```
# Prompt the user for a training haiku file
```

```
training_file = self._prompt_haiku_file("Enter haiku training file: ")
```

```
self.validator.wait_for_enter()
```

```
# Load training data and train the Markov model
```

```
haiku_lines = FileHandler.read_haiku(training_file)
```

```
gen = AIHaikuGenerator()
```

```
gen.train_model(haiku_lines)
```

```
def do_gen():
```

```
    """
```

```
    Generate and display a new haiku using the trained Markov model.
```

```
    The generated haiku is displayed, and the user is given an option to save
```

it

```
    and regenerate if desired.
```

```
    Returns:
```

```
    Optional[None]: Calls itself recursively if the user opts to
    regenerate.
```

```
    """
```

```
    # Generate haiku using AI and display it
```

```
    haiku = gen.generate_haiku()
```

```
    self.display_haiku("AI-Modified Haiku", haiku)
```

```
    # Offer to save and optionally regenerate
```

```
    return self.save_and_redo(haiku, do_gen)
```

```
    return do_gen()
```

```
# ===== #
```

```
# ===== Option 6: Extra Features ===== #
```

```
# ===== #
```

```
def haiku_narrate(self):
```

```
    """
```

```
    Option 6: Narrate a haiku word-by-word using corresponding .wav audio files.
```

```

    This feature reads a haiku from a file and narrates it using pre-recorded
word-by-word
    audio files. The user can optionally save the output as a single .wav file and
repeat
    the narration process.
    """

def perform_narration():
    """
    Handle the process of narrating a haiku.

    Prompts the user for a haiku file, plays audio for each word,
    optionally saves the narration to a file, and allows repeated narration.

    Returns:
        Optional[None]: Calls itself recursively if the user wants to narrate
another haiku.
    """
    # Prompt the user to select a valid haiku file
    haiku_file = self._prompt_haiku_file("Enter the haiku file to narrate: ")
    self.validator.wait_for_enter()

    # Read haiku lines and display them
    haiku_lines = FileHandler.read_haiku(haiku_file)
    self.display_haiku("Haiku to Narrate", haiku_lines)

    # Initialise narrator and perform playback
    narrator = HaikuAudioNarrator(audio_folder="audio_data")
    narrator.narrate(haiku_lines)

    # Ask user if they want to save the narration to a file
    if self.validator.get_yes_or_no("Save audio to file? y/n: ") == "y":
        name = self.validator.prompt_for_new_filename(
            "Enter output audio filename (without .wav): "
        )
        narrator.save_to_audio_file(haiku_lines, name)

    self.validator.wait_for_enter()

    # Ask if user wants to narrate another haiku
    if self.validator.get_yes_or_no("Narrate another haiku? y/n: ") == "y":
        return perform_narration()

    return perform_narration()

```

6.7 Source Code of 'input_utils.py'

```
import os

class InputValidator:
    """
    Utility class for common user input validation tasks.

    This class provides static methods to:
    - Prompt for valid existing files or folders
    - Prompt for a new filename with overwrite confirmation
    - Prompt for yes/no answers with strict validation
    - Pause execution until Enter is pressed

    All methods are static, and the class does not require instantiation.
    It is used throughout the Haikumator application to enforce reliable user input.

    Typical use cases include:
    - File selection and validation
    - Interactive confirmation dialogs
    - Controlled pausing between application states
    """

    # ===== #
    # ===== Prompt for Enter Input ===== #
    # ===== #

    @staticmethod
    def wait_for_enter():
        """
        Prompt the user to press only the Enter key to continue.

        This method pauses the program until the user presses Enter (without any
input).
        It is typically used between major stages to control pacing and readability.

        Returns:
            None
        """
        while True:
            # Wait for the user to press Enter
            confirm = input("\n\nPress Enter to continue...")
            if confirm == "":
                break

    # ===== #
    # ===== Prompt for Yes or No ===== #
```



```

# ===== #

@staticmethod
def get_yes_or_no(prompt):
    """
    Prompt the user for a yes/no response until 'y' or 'n' is entered.

    This method enforces strict input validation and repeats the prompt
    until the user types either 'y' (yes) or 'n' (no), case-insensitively.

    Args:
        prompt (str): The prompt message to display to the user.

    Returns:
        str: The validated response, either 'y' or 'n'.
    """
    while True:
        # Ask the user for input and normalise to lowercase
        response = input(f"\n\n{prompt}").strip().lower()

        # Return only if valid
        if response in ("y", "n"):
            return response

# ===== #
# ===== Prompt for Existing File ===== #
# ===== #

@staticmethod
def prompt_for_existing_file(prompt_message):
    """
    Prompt the user for a filename until a valid existing file is provided.

    This method ensures that the user provides a file path that exists on disk.
    It loops until a non-empty and valid file path is entered.

    Args:
        prompt_message (str): The message displayed to the user when requesting a
filename.

    Returns:
        str: A valid file path that exists on the system.
    """
    while True:
        # Prompt the user to input a file path
        path = input(f"\n\n{prompt_message}").strip()

```

```

    # Reject empty input
    if not path:
        print("No file name entered. Please try again.")
        continue

    # Check if the file actually exists
    if not os.path.isfile(path):
        print("File not found. Please try again.")
        continue

    return path

# ===== #
# ===== Prompt for Folder Name ===== #
# ===== #

@staticmethod
def prompt_for_existing_folder(prompt_message):
    """
    Prompt the user for a folder path until a valid, existing directory is
entered.

    This method ensures that the folder path provided exists on the file system.
    It loops until the user provides a valid, non-empty folder path.

    Args:
        prompt_message (str): The message displayed when prompting the user for
the folder path.

    Returns:
        str: A valid folder path that exists on the system.
    """
    while True:
        # Prompt the user for a folder path
        path = input(f"\n\n{prompt_message}").strip()

        # Check for empty input
        if not path:
            print("No folder name entered. Please try again.")
            continue

        # Check if the path is an existing directory
        if not os.path.isdir(path):
            print("Folder not found. Please try again.")
            continue

    return path

```

```

# ===== #
# ===== Prompt for File Name ===== #
# ===== #

@staticmethod
def prompt_for_new_filename(prompt_message):
    """
    Prompt the user to enter a new filename for saving output.

    If the specified file already exists, the user is asked whether to overwrite
it.
    The loop continues until a valid filename is provided or the user agrees to
overwrite.

    Args:
        prompt_message (str): The message displayed when prompting the user for
input.

    Returns:
        str: A valid filename for writing output.
    """
    while True:
        # Prompt for filename input
        output_file = input(f"{prompt_message}").strip()

        # Reject empty input
        if not output_file:
            print("No file name entered. Please try again.")
            continue

        # Check if file already exists and confirm overwrite
        if os.path.exists(output_file):
            overwrite = InputValidator.get_yes_or_no(
                f"'{output_file}' already exists. Overwrite? y/n: "
            )
            if overwrite == "n":
                continue

        return output_file

```

6.8 Source Code of 'keyword_utils.py'

```

class KeywordIndex:
    """
    An abstract data type (ADT) to track the positions of keywords in tokenized haiku
    lines.

    This structure maps each (line_idx, token_idx) location in the haiku to the list
    of synonyms
    associated with that keyword. It enables efficient batch generation of haiku
    permutations,
    as it allows O(1) access to every location that should be substituted during
    processing.

    Attributes:
        _positions (dict[tuple[int, int], list[str]]): Maps (line index, token index)
                                                    to list of replacement
    synonyms.
    """

    # ===== #
    # ===== KeywordIndex Class Initiation ===== #
    # ===== #

    def __init__(self):
        """
        Initialise an empty KeywordIndex structure.
        """
        # Dictionary mapping (line_idx, token_idx) → list of synonyms
        self._positions = {}

    # ===== #
    # ===== Associate List ===== #
    # ===== #

    def add(self, line_idx: int, token_idx: int, synonyms_list: list[str]):
        """
        Record the position of a keyword token and associate it with its list of
        synonyms.

        Args:
            line_idx (int): The index of the line in the haiku (0-based).
            token_idx (int): The index of the token within the line (0-based).
            synonyms_list (list[str]): List of synonyms for the keyword at the given
        position.

        Returns:
            None

```

```

    """
    self._positions[(line_idx, token_idx)] = synonyms_list

# ===== #
# ===== Iteration of Postition ===== #
# ===== #

def items(self):
    """
    Iterate over all stored positions and their associated synonym lists.

    Yields:
        tuple: A tuple in the form ((line_idx, token_idx), synonyms_list),
               where (line_idx, token_idx) is the position in the haiku,
               and synonyms_list is a list of replacement words.
    """
    for key, syns in self._positions.items():
        yield key, syns

```

6.9 Source Code of 'main.py'

```

"""
Main entry point for the Haikumator application.

This module:
- Displays the welcome banner.
- Launches the main menu loop that allows users to interact with
  haiku transformation features.

Usage:
    Run this file using Anaconda Prompt:
        python main.py
"""

from menu import Menu

def main():
    """
    Launch the Haikumator program by showing the welcome screen
    and entering the main menu loop.
    """
    Menu.welcome()
    Menu.menu()

if __name__ == '__main__':

```

```
main()
```

6.10 Source Code of 'menu.py'

```
from haiku_utils import Haiku
from input_utils import InputValidator

class Menu:
    """
    Handles the display and interaction logic for the Haikumator application menu.

    The Menu class provides static methods to:
    - Display the welcome screen
    - Present the menu options
    - Validate user input
    - Route the user to the appropriate functionality
    - Gracefully handle program exit and pauses

    This class interacts with the main `Haiku` processing engine and allows users
    to explore all transformation features, including synonymizing, zen-izing,
    antonymizing, batch processing, and AI-based haiku generation.

    All methods are static and the class does not require instantiation.
    """

    # ===== #
    # ===== Initial Welcome Page ===== #
    # ===== #

    @staticmethod
    def welcome():
        """
        Display the welcome screen in the exact format specified in the assignment
        brief.

        This method prints a decorative bordered welcome message that includes:
        - Module and application name
        - Author and class details

        The message mimics a "splash screen" and pauses until the user presses Enter.

        Returns:
            None
        """
        border = "*" * 58

        print('\n' + border)
```

```

print("* {:<55}*".format("ST1507 DSAA: Welcome to:"))
print("* {:<55}*".format(""))
print("* {:<55}*".format("~ Haikumator - Haiku Generator Application~"))
print("* {:<55}*".format("-" * 55))
print("* {:<55}*".format(""))
print("* {:<55}*".format("- Done by: Goh Kun Ming(2415691)"))
print("* {:<55}*".format("- Class DAAA/FT/2A/02"))
print(border)

# Pause for user to continue
InputValidator.wait_for_enter()

# ===== #
# ===== Display Choice ===== #
# ===== #

@staticmethod
def display_menu(options):
    """
    Display the list of available menu options to the user.

    The menu is rendered in a readable format where each option is shown
    with its corresponding number and description.

    Args:
        options (dict): A dictionary mapping option numbers (int) to their
        descriptions (str).

    Returns:
        None
    """
    print('\n\nPlease select your choice: (1,2,3,4,5,6,7)')
    for key, label in options.items():
        # Print each menu option in numbered format
        print(f'\t{key}. {label}')

# ===== #
# ===== Choice Validation ===== #
# ===== #

@staticmethod
def get_valid_choice(options):
    """
    Prompt the user for input until a valid menu choice is entered.

    This method ensures robust input handling by:
    - Accepting only digits.

```

- Converting input to an integer.
- Verifying the integer is one of the allowed menu keys.

Args:

options (dict): Dictionary mapping valid integer menu choices to their descriptions.

Returns:

int: A validated menu choice selected by the user.

"""

while True:

choice = input('Enter Choice: ').strip()

Check if input is numeric and within available options

if choice.isdigit() and int(choice) in options:

return int(choice)

Show error and prompt again

print('Please enter a valid number from the menu.')

=====

===== Haikumator Menu =====

=====

@staticmethod

def menu():

"""

Run the main loop for the Haikumator menu system.

This static method:

- Displays the menu to the user.
- Validates the user's menu selection.
- Maps the selection to the corresponding method in the `Haiku` class.
- Continues running until the user chooses to exit.

Returns:

None

"""

haiku = Haiku()

Dictionary of Menu Options (for display)

options = {

1: "Synonymize Haiku",

2: "Zen-ize Haiku",

3: "Antonymize Haiku",

4: "Batch Processing",

5: "AI Modified Haiku",


```

        6: "Narrate Haiku",
        7: "Exit"
    }

    # Mapping user choices to method calls
    action_map = {
        1: haiku.synonymize_haiku,
        2: haiku.zen_ize_haiku,
        3: haiku.antonymize_haiku,
        4: haiku.batch_processing,
        5: haiku.ai_modified_haiku,
        6: haiku.haiku_narrate,
        7: Menu.exit_program
    }

    # Main loop runs until user selects Exit
    while True:
        # Display menu options
        Menu.display_menu(options)

        # Get and validate user's menu selection
        choice = Menu.get_valid_choice(options)

        try:
            # Call the corresponding action
            result = action_map[choice]()

            # Print returned result if any (for debug or extended features)
            if result:
                print(result)
                Menu.wait_for_enter()

        except Exception as e:
            # Catch and report any unexpected errors gracefully
            print(f'An error occurred: {e}')
            Menu.wait_for_enter()

# ===== #
# ===== Exiting of Program ===== #
# ===== #

    @staticmethod
    def exit_program():
        """
        Print the exit message and terminate the program.

        This method is typically called when the user selects the "Exit" option

```

```
from the menu. It cleanly ends the session with a farewell message.
```

```
Returns:
```

```
None
```

```
"""
```

```
# Display farewell message and exit the program
```

```
print('\n\nBye, thanks for using ST1507 DSAA: Haikuator\n\n')
```

```
quit()
```

6.11 Source Code of 'thesaurus.py'

```
import random
```

```
class Thesaurus:
```

```
    """
```

```
    A class to manage synonyms and antonyms loaded from text files.
```

```
    The Thesaurus class provides:
```

- Dictionary-based storage for synonyms and antonyms.
- Efficient lookup of random or shortest synonyms for keywords.
- Reverse mapping to retrieve antonyms even when a synonym is passed.

```
    It supports loading structured data from text files where each line is formatted as:
```

```
        keyword: synonym1, synonym2, ...
```

```
        keyword: antonym1, antonym2, ...
```

```
    This class is integral to haiku transformations such as:
```

- Synonymize Haiku
- Zen-ize Haiku
- Antonymize Haiku

```
    Attributes:
```

```
        synonyms (dict[str, list[str]]): Maps keywords to lists of synonyms.
```

```
        antonyms (dict[str, list[str]]): Maps keywords to lists of antonyms.
```

```
    """
```

```
# ===== #
```

```
# ===== Thesaurus Class Initiation ===== #
```

```
# ===== #
```

```
def __init__(self):
```

```
    """
```

```
    Initialise an empty Thesaurus instance.
```

```
    This constructor sets up two dictionaries:
```

- `synonyms`: Maps a keyword to a list of its synonyms.

```

- `antonyms`: Maps a keyword to a list of its antonyms.

These dictionaries are populated via `load_synonyms()` and `load_antonyms()`.

Attributes:
    synonyms (dict[str, list[str]]): Stores synonym lists by keyword.
    antonyms (dict[str, list[str]]): Stores antonym lists by keyword.
"""
# Dictionary to hold keyword → list of synonyms
self.synonyms = {}

# Dictionary to hold keyword → list of antonyms
self.antonyms = {}

# ===== #
# ===== Load Input Synonyms File ===== #
# ===== #

def load_synonyms(self, file_path):
    """
    Load synonym data from a thesaurus file into the `synonyms` dictionary.

    Each line in the file must follow the format:
        keyword: synonym1, synonym2, ...

    The method clears any existing synonym data before loading new entries.
    Both the keyword and synonyms are stripped of leading/trailing whitespace
    and stored in lowercase for consistent lookup.

    Args:
        file_path (str): Path to the synonym thesaurus text file.

    Returns:
        None
    """
    # Clear any existing synonym data
    self.synonyms.clear()

    with open(file_path, 'r') as f:
        for line in f:
            # Process only lines with the expected key-value structure
            if ':' in line:
                keyword, syms = line.strip().split(":")
                # Populate the dictionary with cleaned keyword and list of
synonyms
                self.synonyms[keyword.strip()] = [s.strip() for s in
syns.split(",")]

```

```

# ===== #
# ===== Load Input Antonyms File ===== #
# ===== #

def load_antonyms(self, file_path):
    """
    Load antonym data from a thesaurus file into the `antonyms` dictionary.

    Each line in the file must follow the format:
        keyword: antonym1, antonym2, ...

    The method clears any existing antonym data before loading new entries.
    Keywords and antonyms are stripped of surrounding whitespace and stored in
lowercase.

    Args:
        file_path (str): Path to the antonym thesaurus text file.

    Returns:
        None
    """
    # Clear existing antonyms before loading new ones
    self.antonyms.clear()

    with open(file_path, 'r') as f:
        for line in f:
            # Process only valid lines containing a colon
            if ':' in line:
                keyword, ants = line.strip().split(":")
                # Store the antonyms as a list under the cleaned keyword
                self.antonyms[keyword.strip()] = [a.strip() for a in
ants.split(",")]

# ===== #
# ===== Get Synonym for Word ===== #
# ===== #

def get_random_synonym(self, word):
    """
    Return a random synonym for the given keyword.

    This method performs an O(1) dictionary lookup to find the list of synonyms
for the given keyword. It then selects one synonym randomly.

    Args:
        word (str): The keyword to look up in the synonym dictionary.

```

```

    Returns:
        str or None: A randomly selected synonym, or None if the keyword is not
found.
    """
    word = word.lower()

    # Check if the keyword exists in the synonym dictionary
    if word in self.synonyms:
        return random.choice(self.synonyms[word])

    # Return None if keyword not found
    return None

# ===== #
# ===== Shortest Synonym Word ===== #
# ===== #

def get_shortest_synonym(self, word):
    """
    Return the shortest synonym for the given keyword.

    This method is particularly useful for the Zen-ize transformation of haikus,
    where the goal is to make lines more compact by replacing words with the
    shortest available synonym. If multiple synonyms share the shortest length,
    one is selected randomly.

    Args:
        word (str): The keyword to look up in the synonym dictionary.

    Returns:
        str or None: A randomly chosen shortest synonym, or None if the keyword is
not found.
    """
    word = word.lower()

    if word in self.synonyms:
        synonym_list = self.synonyms[word]

        # Find the minimum length among all synonyms
        min_length = min(len(s) for s in synonym_list)

        # Filter all synonyms that match the minimum length
        shortest = [s for s in synonym_list if len(s) == min_length]

        # Return a randomly selected synonym from the shortest list
        return random.choice(shortest)

```

```

    # Return None if the keyword is not in the synonym dictionary
    return None

# ===== #
# ===== Get Antonym for Word ===== #
# ===== #

def get_random_antonym(self, word_or_synonym):
    """
    Retrieve a random antonym for a given word or one of its known synonyms.

    This method supports two types of lookup:
    1. Direct: If the input word is a key in the antonym dictionary, a random
    antonym is returned.
    2. Reverse: If the input is a known synonym (but not a keyword), the method
    attempts to find
        the original keyword in the synonym list and then retrieve its antonym.

    Args:
        word_or_synonym (str): The word to find an antonym for. It can be either a
    keyword
        or a known synonym of a keyword.

    Returns:
        str or None: A randomly selected antonym if found; otherwise, None.
    """
    word_or_synonym = word_or_synonym.lower()

    # Direct Antonym Lookup
    if word_or_synonym in self.antonyms:
        return random.choice(self.antonyms[word_or_synonym])

    # Reverse Lookup: Check if word is a synonym of a keyword
    for keyword, syn_list in self.synonyms.items():
        if word_or_synonym in syn_list and keyword in self.antonyms:
            return random.choice(self.antonyms[keyword])

    # No antonym found
    return None

```

6.12 Source Code of ‘tokenized_utils.py’

```

class TokenizedLine:
    """
    Represents a single haiku line tokenized into (prefix, core, suffix) tuples.

```

This structure enables fine-grained manipulation of individual words in a line, including the ability to preserve punctuation before and after each word. It is particularly useful when performing transformations such as replacing keywords with synonyms or antonyms while retaining original formatting.

Attributes:

`raw_line (str)`: The original unmodified haiku line.

`tokens (list[tuple[str, str, str]])`: A list of tuples in the form (prefix, core, suffix),

where 'prefix' and 'suffix' are punctuations and

'core' is the central word.

"""

```
# ===== #
# ===== TokenizedLine Class Initiation ===== #
# ===== #
```

```
def __init__(self, raw_line: str):
    """
```

Initialise the TokenizedLine object by splitting and analysing the input line.

Args:

`raw_line (str)`: A single line from a haiku.

"""

```
self.raw_line = raw_line
```

```
self.tokens = []
```

```
# Tokenize each word into (prefix, core, suffix)
```

```
for word in raw_line.split():
```

```
    prefix, core, suffix = "", word, ""
```

```
    # Extract Leading Punctuation
```

```
    while core and core[0] in ".,:;?!\"'":
```

```
        prefix += core[0]
```

```
        core = core[1:]
```

```
    # Extract Trailing Punctuation
```

```
    while core and core[-1] in ".,:;?!\"'":
```

```
        suffix = core[-1] + suffix
```

```
        core = core[:-1]
```

```
    # Store the token
```

```
    self.tokens.append((prefix, core, suffix))
```

```
# ===== #
# ===== Reconstruct Tozenized Parts ===== #
```

```

# ===== #

def reconstruct(self) -> str:
    """
    Reconstruct the line from its tokenized components.

    Returns:
        str: The reconstructed line with spacing and punctuation preserved.
    """
    return " ".join(f"{p}{c}{s}" for (p, c, s) in self.tokens)

# ===== #
# ===== Copy of Tozenized Line ===== #
# ===== #

def copy(self):
    """
    Create a deep copy of this TokenizedLine instance.

    Returns:
        TokenizedLine: A new TokenizedLine object with an identical token
structure.
    """
    clone = TokenizedLine(self.raw_line)
    clone.tokens = [(p, c, s) for (p, c, s) in self.tokens]
    return clone

```

7. References

Russel, S. and Norvig, P. (2020). *Artificial intelligence: A Modern approach*. 4th ed. Prentice Hall.

Cormen, T., Leiserson, C., Rivest, R. and Stein, C. (2009). *Introduction to Algorithms*. Third ed. Cambridge, Mass.: Mit Press.

Mitzenmacher, M., Upfal, E. and Cambridge University Press (2018). *Probability and computing : randomization and probabilistic techniques in algorithms and data analysis*. Cambridge: Cambridge University Press.